

Unidad 2: Distintos modelos NoSQL

Clase 4 – Diseño documental y analítica de datos aplicada

Objetivo

Aplicar técnicas avanzadas de modelado y consulta en bases de datos documentales, utilizando agregaciones e índices para resolver problemáticas reales de análisis de datos y generación de hallazgos.

Tema

- MongoDB / MQL avanzado
 - Agregaciones
 - Índices
 - Diseño orientado a consultas
 - Casos de auditoría (hallazgos, evidencia, eventos)
- Ejercitación aplicada al TP.

Diseño documental y analítica de datos aplicada en MongoDB

En esta clase vamos a dar un paso importante dentro del trabajo con bases de datos documentales. Hasta ahora, el foco estuvo en comprender qué es MongoDB, cómo se almacenan los documentos en formato JSON y de qué manera se realizan operaciones básicas de consulta y manipulación. A partir de este punto, el objetivo cambia de escala. Ya no se trata solamente de guardar datos, sino de pensar cómo esos datos pueden organizarse, consultarse y analizarse para responder preguntas reales del negocio. En otras palabras, comenzamos a trabajar con MongoDB no solo como una base de datos documental, sino también como una herramienta capaz de apoyar procesos de análisis, investigación y generación de hallazgos.

Este cambio es fundamental porque modifica la lógica con la que muchas veces se diseña una base de datos. Quienes vienen del mundo relacional suelen pensar primero en entidades, relaciones, normalización y tablas separadas. En MongoDB, en cambio, el diseño se construye desde otra pregunta: ¿qué consultas necesito resolver? La estructura documental no se define únicamente por cómo “se ve” el negocio, sino también por cómo se va a explotar la información. A esto lo llamamos diseño orientado a consultas.

Supongamos, por ejemplo, un caso de auditoría de eventos. En un entorno corporativo pueden registrarse acciones como accesos al sistema, cambios de permisos, aprobaciones de pagos, modificaciones de proveedores o eliminación de registros sensibles. Si quisiéramos analizar estos hechos con una lógica tradicional, quizás pensaríamos en varias tablas vinculadas entre sí. Pero si el objetivo real es detectar patrones, analizar secuencias, encontrar comportamientos fuera de horario o agrupar

actividades por usuario, entonces lo más importante no es la relación formal entre entidades, sino la forma en que esos datos estarán disponibles para consulta rápida y análisis posterior. Allí aparece con fuerza la idea de diseñar documentos pensando en el uso analítico que tendrán.

Del diseño por entidades al diseño por preguntas analíticas

En bases relacionales, el diseño suele empezar por identificar entidades como Usuario, Evento, Área, Hallazgo o Evidencia, y luego establecer relaciones entre ellas. Esa lógica tiene muchas ventajas en contextos transaccionales y altamente estructurados. Sin embargo, en una base documental como MongoDB, el desafío es otro. Aquí conviene pensar en colecciones y documentos que agrupen la información de una forma útil para responder preguntas concretas.

Esto no significa que se ignore el negocio, sino que se modela desde una perspectiva distinta. Si una organización necesita analizar eventos de seguridad, quizás sea más eficiente almacenar un documento por proceso, por sesión o por caso auditado, con los eventos incrustados dentro del documento, en lugar de separar todo en múltiples colecciones y reconstruir luego la información mediante operaciones complejas. El criterio principal es facilitar la lectura, el análisis y la velocidad de acceso.

Imaginemos un documento como el siguiente:

```
{
  "proceso": "Pagos a proveedores",
  "area": "Finanzas",
  "responsable": "Laura Gómez",
  "eventos": [
    {
      "usuario": "lgomez",
      "tipo": "aprobacion",
      "fecha": "2026-04-10T09:15:00",
      "monto": 120000
    },
    {
      "usuario": "lgomez",
      "tipo": "modificacion",
      "fecha": "2026-04-10T22:48:00",
      "monto": 120000
    }
  ]
}
```

En este ejemplo, el documento no se diseñó únicamente para representar entidades, sino para permitir consultas analíticas sobre un proceso y sus eventos asociados. Si un auditor quiere detectar actividades fuera de horario, o analizar qué usuarios

concentran mayor cantidad de modificaciones, la información ya está agrupada de una forma útil para responder esas preguntas.

Aquí aparece una diferencia central con el paradigma relacional. En SQL, muchas veces el diseño apunta a reducir redundancia y mantener integridad estructural. En MongoDB, en cambio, el diseño prioriza la eficiencia de lectura para determinados casos de uso. Por eso se dice que el modelado documental se orienta a consultas y no exclusivamente a entidades.

MongoDB como herramienta analítica

Un error frecuente es pensar que MongoDB sirve solo para almacenar documentos flexibles, pero no para hacer análisis. En realidad, MongoDB posee capacidades analíticas muy potentes, especialmente a través del pipeline de agregación. Este mecanismo permite transformar, filtrar, agrupar, ordenar y resumir datos en varias etapas encadenadas. El resultado es un flujo de trabajo muy adecuado para preguntas del tipo: cuántos eventos hubo por usuario, qué áreas presentan más hallazgos, cuáles son los montos totales aprobados fuera de horario, o qué evidencias están asociadas a procesos críticos.

La lógica de un pipeline puede compararse con una línea de producción. Los documentos entran por una etapa inicial, pasan por distintos procesos y salen transformados al final. Cada etapa tiene una función específica. Algunas filtran documentos, otras cambian su forma, otras agrupan y calculan resultados. Lo importante es entender que no se trata de una consulta plana, sino de una secuencia de operaciones encadenadas que permiten construir análisis cada vez más complejos.

Pipeline de agregación: concepto y propósito

El pipeline de agregación de MongoDB se ejecuta con el método `aggregate()`. Dentro de este método se define un arreglo de etapas, donde cada etapa comienza con un operador especial precedido por el signo `$`. Cada operador cumple una tarea específica dentro del flujo.

El valor de este mecanismo está en que permite trabajar de forma progresiva. En lugar de escribir una única consulta difícil de interpretar, el análisis se construye paso a paso. Esto favorece la comprensión, la depuración y el diseño lógico del razonamiento analítico.

Supongamos que una colección llamada `eventos_auditoria` contiene documentos como estos:

```
{
  "usuario": "jruiz",
  "area": "Finanzas",
  "tipo": "aprobacion",
  "fecha": "2026-04-15T10:30:00",
```

```
"monto": 250000
}

{
  "usuario": "jruiz",
  "area": "Finanzas",
  "tipo": "modificacion",
  "fecha": "2026-04-15T23:10:00",
  "monto": 250000
}

{
  "usuario": "mperez",
  "area": "Compras",
  "tipo": "alta_proveedor",
  "fecha": "2026-04-15T14:20:00",
  "monto": 0
}
```

A partir de estos documentos, podríamos construir distintas preguntas analíticas. Por ejemplo, cuántos eventos generó cada usuario, qué áreas concentran más actividad o qué operaciones superan determinado monto.

La etapa \$match: filtrar antes de analizar

La etapa \$match se utiliza para seleccionar únicamente los documentos que cumplen ciertas condiciones. Su función es parecida al WHERE en SQL. Es muy importante porque permite reducir el volumen de datos antes de pasar a etapas más complejas del pipeline.

Por ejemplo, si queremos analizar solo eventos del área de Finanzas, la consulta podría ser:

```
db.eventos_auditoria.aggregate([
  { $match: { area: "Finanzas" } }
])
```

En este caso, solo se procesarán los documentos cuya área sea Finanzas. Esto mejora la claridad del análisis y, además, puede mejorar el rendimiento si la colección es grande.

En auditoría, \$match resulta especialmente útil para enfocar investigaciones. Se puede usar para aislar eventos de un usuario, filtrar acciones realizadas en una fecha determinada o seleccionar operaciones por tipo de movimiento. La lógica es sencilla: primero acotar el universo; después analizar.

La etapa \$group: agrupar y resumir información

La etapa \$group es una de las más potentes del pipeline. Permite reunir documentos según un criterio común y generar resultados resumidos, como conteos, sumas, promedios, máximos o mínimos. Si \$match sirve para seleccionar, \$group sirve para sintetizar.

Si quisiéramos contar cuántos eventos generó cada usuario, podríamos escribir:

```
db.eventos_auditoria.aggregate([
  {
    $group: {
      _id: "$usuario",
      cantidad_eventos: { $sum: 1 }
    }
  }
])
```

Aquí, MongoDB toma todos los documentos de la colección, los agrupa por el campo usuario y cuenta cuántos documentos hay en cada grupo. El resultado ya no será una lista de eventos individuales, sino una lista resumida con la cantidad de eventos por usuario.

Este tipo de consulta es muy valioso en contextos reales. En un escenario de auditoría, puede ayudar a detectar concentración de actividad en determinados usuarios, comportamiento inusual o acumulación de operaciones en áreas críticas. También puede servir para construir indicadores, como cantidad de evidencias por proceso o cantidad de hallazgos por tipo de control.

La etapa \$project: redefinir la forma de salida

La etapa \$project permite seleccionar qué campos se mostrarán en la salida y también crear nuevos campos derivados. Su función se parece al SELECT de SQL, aunque ofrece mayor flexibilidad porque puede reformular la estructura del documento resultante.

Por ejemplo, si queremos mostrar usuario, tipo y monto, pero omitir otros campos, podemos hacer:

```
db.eventos_auditoria.aggregate([
  {
    $project: {
      _id: 0,
      usuario: 1,
      tipo: 1,
      monto: 1
    }
  }
])
```

Esto devuelve una salida más limpia y enfocada. También se puede usar para renombrar campos o construir campos calculados. En análisis de datos, esta etapa es útil para preparar resultados finales comprensibles, especialmente cuando luego esos datos serán exportados o revisados por otras personas.

La etapa \$sort: ordenar para interpretar mejor

Cuando ya se obtuvieron resultados resumidos, muchas veces es necesario ordenarlos. La etapa \$sort permite hacerlo de manera ascendente o descendente.

Si retomamos el ejemplo de cantidad de eventos por usuario, podemos ordenar de mayor a menor:

```
db.eventos_auditoria.aggregate([
  {
    $group: {
      _id: "$usuario",
      cantidad_eventos: { $sum: 1 }
    }
  },
  {
    $sort: { cantidad_eventos: -1 }
  }
])
```

Este tipo de ordenamiento es fundamental en cualquier análisis. No basta con saber los valores; muchas veces es necesario identificar rápidamente los más altos, los más bajos o los más relevantes. En una revisión de auditoría, por ejemplo, ordenar puede permitir detectar de inmediato al usuario con mayor volumen de actividad o al área con mayor número de excepciones.

La etapa \$limit: enfocarse en los casos más importantes

Una vez ordenados los resultados, puede resultar útil quedarse solo con los primeros registros. La etapa \$limit permite hacer esto.

Si queremos ver los tres usuarios con mayor cantidad de eventos, podríamos escribir:

```
db.eventos_auditoria.aggregate([
  {
    $group: {
      _id: "$usuario",
      cantidad_eventos: { $sum: 1 }
    }
  },
  {
    $sort: { cantidad_eventos: -1 }
  },
  {
    $limit: 3
  }
])
```

```
}
{
  $limit: 3
}
})
```

Este enfoque es muy habitual en análisis de negocio. Se utiliza para construir rankings, detectar casos prioritarios o seleccionar muestras relevantes. En lugar de trabajar con todo el universo, se focaliza la atención en los casos más significativos.

La etapa \$unwind: analizar arreglos internos

Una característica frecuente del modelo documental es que un documento puede contener arreglos. Esto es muy útil para representar eventos, evidencias, comentarios o movimientos asociados a un mismo caso. Sin embargo, cuando se quiere analizar cada elemento del arreglo de manera individual, es necesario “desarmar” esa estructura. Para eso se usa \$unwind.

Supongamos que una colección almacena procesos con un arreglo de eventos:

```
{
  "proceso": "Pago a proveedores",
  "eventos": [
    { "usuario": "lgomez", "tipo": "aprobacion" },
    { "usuario": "lgomez", "tipo": "modificacion" },
    { "usuario": "mlopez", "tipo": "validacion" }
  ]
}
```

Si queremos contar los eventos por tipo, primero necesitamos convertir cada elemento del arreglo en un documento independiente dentro del pipeline. Eso se logra así:

```
db.procesos.aggregate([
  { $unwind: "$eventos" },
  {
    $group: {
      _id: "$eventos.tipo",
      cantidad: { $sum: 1 }
    }
  }
])
```

\$unwind toma el arreglo eventos y lo expande. Cada elemento se procesa como si fuera una fila independiente para las siguientes etapas del pipeline. Esta operación resulta clave cuando se trabaja con documentos embebidos y se necesita analizar su contenido interno.

Pensar consultas reales antes de modelar

Hasta aquí puede verse que MongoDB ofrece herramientas muy potentes para el análisis. Pero estas herramientas solo se aprovechan de verdad cuando el diseño documental fue pensado desde las consultas reales. Aquí aparece uno de los conceptos más importantes de la clase: no se modela solo para guardar, sino para consultar bien.

Diseñar orientado a consultas implica hacerse preguntas antes de crear la estructura. ¿Qué información voy a buscar con mayor frecuencia? ¿Qué análisis deberá responder el sistema? ¿Qué campos se usarán en filtros, agrupaciones u ordenamientos? ¿Qué relaciones conviene embeber y cuáles conviene separar? La respuesta a estas preguntas define la forma del documento.

Por ejemplo, si en un sistema de auditoría lo más habitual será consultar todos los eventos asociados a un proceso o a un caso, entonces tiene sentido embeber esos eventos dentro del documento principal. De esa manera, al recuperar un caso ya se obtiene toda la secuencia asociada. En cambio, si los eventos son extremadamente numerosos, crecen constantemente y además se analizan por separado en grandes volúmenes, entonces puede convenir una colección independiente.

No existe una única respuesta universal. La clave está en comprender el uso que tendrá la información. MongoDB exige pensar desde el patrón de acceso.

Diseño orientado a consultas y minimización de joins

Uno de los principios más conocidos del modelado documental es que se busca minimizar la necesidad de joins. En bases relacionales, los joins son parte natural del diseño. En MongoDB, aunque existen mecanismos como \$lookup, el mejor rendimiento suele alcanzarse cuando la estructura documental evita reconstrucciones frecuentes entre colecciones.

Si una consulta necesita permanentemente leer proceso, usuario, área y eventos asociados, lo más eficiente es considerar un diseño donde esa información ya esté cercana dentro del mismo documento o en una estructura fácil de resolver. Esto no significa duplicar datos de forma descontrolada, sino hacerlo con criterio. En entornos analíticos, una redundancia moderada puede ser aceptable si mejora radicalmente la velocidad y la simplicidad de consulta.

La idea no es copiar datos porque sí, sino diseñar con una intención analítica. En muchos sistemas reales, el valor no está en mantener una pureza estructural absoluta, sino en responder preguntas de forma rápida, clara y escalable.

Índices: acelerar la búsqueda y mejorar el rendimiento

Una vez entendido el valor del diseño orientado a consultas, aparece otro elemento central: los índices. Un índice es una estructura auxiliar que MongoDB crea para

acceder más rápidamente a ciertos documentos. Su función es similar al índice de un libro: en lugar de recorrer página por página hasta encontrar un tema, se puede ir directamente a la ubicación relevante.

Cuando una colección no tiene índice sobre el campo consultado, MongoDB debe recorrer todos los documentos para encontrar coincidencias. A esto se lo suele llamar escaneo completo de colección. En conjuntos pequeños, quizá no se note la diferencia. Pero en colecciones grandes, esto puede volver las consultas lentas y costosas.

Por ejemplo, si una organización registra millones de eventos y se consulta frecuentemente por usuario, conviene crear un índice sobre ese campo:

```
db.eventos_auditoria.createIndex({ usuario: 1 })
```

A partir de ese momento, las búsquedas por usuario serán mucho más eficientes. Lo mismo ocurre con campos como fecha, tipo de evento o área, siempre que se usen con frecuencia en filtros.

Si además las consultas combinan más de un campo, puede ser útil un índice compuesto. Por ejemplo:

```
db.eventos_auditoria.createIndex({ area: 1, fecha: -1 })
```

Este índice resulta útil cuando es frecuente consultar eventos de un área específica y luego ordenar o filtrar por fecha. La clave está en entender que los índices deben responder a patrones de consulta concretos. No se crean de manera arbitraria.

El costo de los índices y su uso inteligente

Aunque los índices mejoran las búsquedas, también tienen un costo. Ocupan espacio y deben actualizarse cuando se insertan, modifican o eliminan documentos. Por eso no conviene indexar todo indiscriminadamente. El criterio correcto es identificar los campos más usados en búsquedas, filtros, ordenamientos y análisis recurrentes.

En términos prácticos, conviene hacerse estas preguntas: ¿sobre qué campos se consulta con más frecuencia? ¿Qué filtros aparecen todo el tiempo? ¿Qué consultas tardarían demasiado si la colección creciera mucho? La respuesta ayuda a decidir qué índices tienen sentido.

En un sistema de auditoría, por ejemplo, suele ser razonable indexar campos como usuario, fecha, tipo de evento o identificador de proceso. En cambio, quizás no tenga sentido indexar un campo que casi nunca se consulta. El objetivo no es tener muchos índices, sino los adecuados.

Caso aplicado: auditoría, hallazgos, evidencia y eventos

Para integrar todos estos conceptos, conviene pensar un escenario aplicado. Imaginemos un sistema de auditoría interna que registra eventos de control dentro de

distintos procesos organizacionales. Cada evento puede representar una acción realizada por un usuario, una aprobación, una modificación, una anulación, un cambio de permisos o una carga de información sensible. A partir de esos registros, el equipo de auditoría busca detectar anomalías, construir evidencia y generar hallazgos.

Un hallazgo no surge de la nada. Surge del análisis de información. Por ejemplo, puede detectarse que un mismo usuario aprobó y luego modificó una operación crítica fuera del horario habitual. También podría observarse que un proveedor fue dado de alta y utilizado en pagos en un intervalo demasiado corto. O que un área presenta una concentración anormal de excepciones. En todos estos casos, MongoDB puede actuar como soporte para el almacenamiento y para el análisis exploratorio.

Supongamos una colección de eventos con esta lógica:

```
{
  "proceso": "Compras",
  "usuario": "jruiz",
  "area": "Abastecimiento",
  "tipo": "alta_proveedor",
  "fecha": "2026-04-17T23:30:00",
  "detalle": {
    "proveedor": "Soluciones Delta SA",
    "pais": "Argentina"
  }
}
```

A partir de este tipo de estructura, pueden formularse preguntas analíticas relevantes. ¿Qué usuarios realizaron acciones fuera de horario? ¿Qué procesos presentan mayor concentración de cambios? ¿Qué área registra más eventos críticos? ¿Cuántos eventos vinculados a altas de proveedores ocurrieron en el mes? ¿Qué casos ameritan una revisión más profunda?

Estas preguntas son valiosas porque muestran a los estudiantes que una base documental no es solo una tecnología de almacenamiento, sino un instrumento para construir conocimiento sobre la operación.

Ejemplos de análisis aplicados a auditoría

Si se quisiera detectar cantidad de eventos por área, la consulta podría ser:

```
db.eventos_auditoria.aggregate([
  {
    $group: {
      _id: "$area",
      cantidad_eventos: { $sum: 1 }
    }
  },
])
```

```
{
  $sort: { cantidad_eventos: -1 }
}
])
```

Si el interés fuera identificar tipos de evento más frecuentes:

```
db.eventos_auditoria.aggregate([
  {
    $group: {
      _id: "$tipo",
      cantidad: { $sum: 1 }
    }
  },
  {
    $sort: { cantidad: -1 }
  }
])
```

Y si se quisiera analizar eventos de un usuario específico:

```
db.eventos_auditoria.aggregate([
  { $match: { usuario: "jruiz" } },
  {
    $project: {
      _id: 0,
      usuario: 1,
      tipo: 1,
      fecha: 1,
      area: 1
    }
  },
  { $sort: { fecha: 1 } }
])
```

Cada uno de estos ejemplos responde a una necesidad concreta. No son consultas “por practicar”, sino análisis que podrían formar parte de una investigación real.

De los datos a los hallazgos

Uno de los aprendizajes más importantes de esta clase es comprender que el análisis de datos no consiste solamente en ejecutar consultas. Analizar significa interpretar. Los datos son el punto de partida, pero el valor surge cuando esos resultados permiten formular conclusiones.

Si una consulta muestra que un usuario tiene una cantidad inusualmente alta de modificaciones, eso no implica automáticamente fraude o error. Pero sí constituye una

señal que puede justificar revisión. Si un área concentra la mayoría de los eventos críticos, quizá corresponda analizar controles internos, segregación de funciones o distribución de responsabilidades. El hallazgo nace cuando el dato se contextualiza y se convierte en evidencia de una situación relevante.

Este enfoque resulta especialmente potente para estudiantes, porque conecta la tecnología con la realidad profesional. MongoDB no aparece solo como lenguaje o motor, sino como soporte para procesos de observación, control, investigación y toma de decisiones.

Cómo llevar estos conceptos al trabajo práctico

Todo lo visto en esta clase puede trasladarse directamente al TP. Cada grupo puede pensar su caso de negocio y definir qué preguntas analíticas necesita resolver. No alcanza con describir documentos o mostrar colecciones. El verdadero valor aparece cuando pueden justificar por qué modelaron de cierta manera y cómo esa estructura les permite responder preguntas relevantes.

Si un grupo trabaja con un caso de auditoría, podría definir preguntas como estas: qué usuario registró más actividad, qué procesos concentran más eventos, qué áreas presentan más hallazgos potenciales o qué tipo de evento aparece con mayor frecuencia. A partir de esas preguntas, deberían diseñar sus documentos, construir agregaciones y decidir qué índices resultan razonables.

Si otro grupo trabaja con logística, retail o atención al cliente, la lógica es la misma. Siempre deben comenzar por el uso analítico esperado. El diseño documental se vuelve sólido cuando está conectado con necesidades concretas de consulta.

Cierre conceptual de la clase

La enseñanza principal de esta clase es que MongoDB debe entenderse desde una doble dimensión. Por un lado, como una base de datos documental flexible, capaz de representar estructuras complejas con naturalidad. Por otro, como una plataforma que permite analizar esos datos para responder preguntas reales. En ese cruce entre modelado y análisis aparece el verdadero potencial de la tecnología.

Diseñar orientado a consultas implica pensar antes de modelar. Significa anticipar preguntas, comprender patrones de acceso y estructurar documentos de manera útil para la lectura y el análisis. El pipeline de agregación, por su parte, permite transformar los datos en información resumida e interpretable. Los índices completan el esquema, asegurando que las consultas frecuentes sean eficientes incluso cuando el volumen crece.

En contextos de auditoría, control, monitoreo o análisis operativo, esta combinación resulta especialmente valiosa. Permite registrar eventos, analizarlos, encontrar patrones y convertir datos dispersos en hallazgos relevantes.